

Subsetting Data in R

Data Wrangling in R

Overview

We showed different ways to read data into R using:

```
readr::read_csv()  
readr::read_delim()  
readxl::read_excel()
```

In this module, we will show you how select rows and columns of datasets.

Setup

We will be using the **dplyr** package in the tidyverse.

Here are several resources on how to use **dplyr**:

- <https://dplyr.tidyverse.org/>
- <https://r4ds.had.co.nz/>
- <https://cran.rstudio.com/web/packages/dplyr/vignettes/dplyr.html>
- <https://stat545.com/dplyr-intro.html>

The **dplyr** package also interfaces well with tibbles.

Dataset

We will be using some UFO data from the <https://nuforc.org/> that was previously available from Kaggle (<https://www.kaggle.com/>).

We will deal with the problem indicated here soon.

```
library(tidyverse)
ufo <- read_csv(
  "https://sisbid.github.io/Data-Wrangling/data/ufo/ufo_data_complete.csv")
```

Warning: One or more parsing issues, call `problems()` on your data frame for details,

e.g.:

```
dat <- vroom(...)
problems(dat)
```

Selecting a single column of a **data.frame**:

To grab just the values from a single column, you would use the `pull` function. The output will be a vector (and not a **tibble**).

Since this is a long vector we will just show the first 6 values using the `head` function around the output of the `pull` function.

```
head(pull(ufo, country))
```

```
[1] "us" NA    "gb" "us" "us" "us"
```

Using the **pipe** (comes with **dplyr**):

That was a lot of typing and nested functions, which can be confusing. The pipe `%>%` or (you will also see this pipe `|>`) makes things such as this much more readable. It reads left side “pipes” into right side. RStudio **CMD/Ctrl + Shift + M** shortcut.

Using the **pipe** (comes with **dplyr**):

Pipe `ufo` into `select`, then pipe that into `pull`, and then show the head:

```
ufo %>% pull(country) %>% head()
```

```
[1] "us" NA    "gb" "us" "us" "us"
```

Selecting a single column of a **data.frame**:

The `pull` function is equivalent to using the `$` method (in base R).

Note that base R and tidyverse don't always play nice together.

```
head(pull(ufo, country))
```

```
[1] "us" NA  "gb" "us" "us" "us"
```

```
head(ufo$country)
```

```
[1] "us" NA  "gb" "us" "us" "us"
```

Note this does *not* return a **tibble** (or **data.frame**) but rather a vector.

Selecting a single column of a `data.frame`:

The `select` function extracts one or more columns from a `tibble` or `data.frame` and returns a `tibble` (not a vector).

```
select(ufo, country)
```

```
# A tibble: 88,875 × 1
  country
  <chr>
1 us
2 <NA>
3 gb
4 us
5 us
6 us
7 gb
8 us
9 us
10 us
# i 88,865 more rows
```

Selecting multiple columns of a `data.frame`:

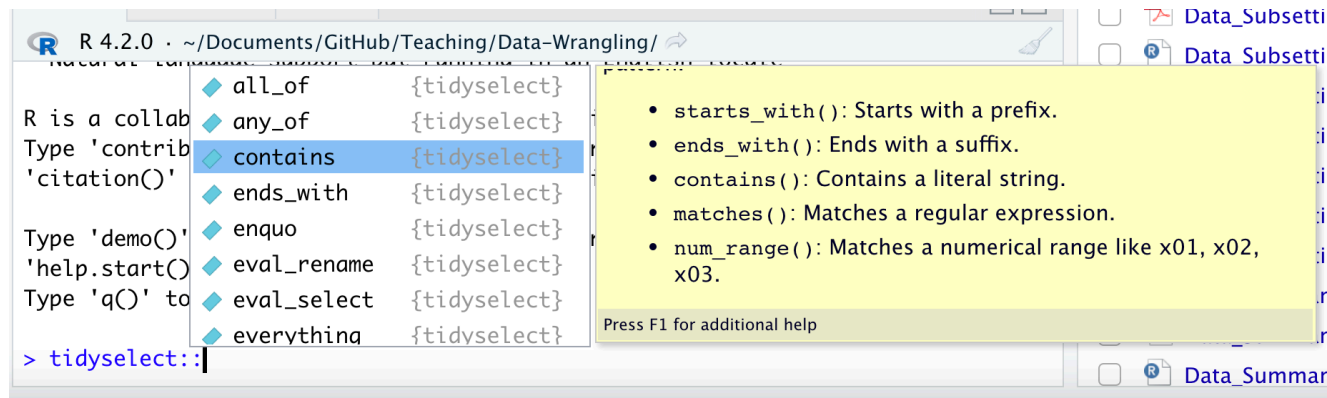
The `select` command from `dplyr` is very flexible. You just need to list all columns you want to extract separated by commas. You can use this as a way to just keep the columns you want for example.

```
select(ufo, country, shape)
```

```
# A tibble: 88,875 × 2
  country shape
  <chr>    <chr>
1 us      cylinder
2 <NA>     light
3 gb      circle
4 us      circle
5 us      light
6 us      sphere
7 gb      circle
8 us      disk
9 us      disk
10 us     disk
# i 88,865 more rows
```

See the Select “helpers”

Type `tidyselect::` to see functions available.



Here are a few:

```
last_col()
ends_with()
starts_with()
contains() # search for a pattern
everything()
```

Tidymodel helpers

For example, we can take all columns that start with a "c":

```
ufo %>% select(starts_with("c"))
```

```
# A tibble: 88,875 × 3
```

	city	country	comments
	<chr>	<chr>	<chr>
1	san marcos	us	This event took place in early fall around 1949...
2	lackland afb	<NA>	1949 Lackland AFB TX. Lights racing across...
3	chester (uk/england)	gb	Green/Orange circular disc over Chester TX Eng...
4	edna	us	My older brother and twin sister were leaving t...
5	kaneohe	us	AS a Marine 1st Lt. flying an FJ4B fighter/atta...
6	bristol	us	My father is now 89 my brother 52 the girl with...
7	penarth (uk/wales)	gb	penarth uk circle 3mins stayed 30ft above me...
8	norwalk	us	A bright orange color changing to reddish color...
9	pell city	us	Strobe Lighted disk shape object observed close...
10	live oak	us	Saucer zaps energy from powerline as my pregnan...

```
# i 88,865 more rows
```

Tidysselect helpers

Or we can take all columns that end with an "e":

```
ufo %>% select(ends_with("e"))
```

```
# A tibble: 88,875 × 5
```

	datetime	state	shape	latitude	longitude
	<chr>	<chr>	<chr>	<chr>	<chr>
1	10/10/1949 20:30	tx	cylinder	29.8830556	-97.9411111
2	10/10/1949 21:00	tx	light	29.38421	-98.581082
3	10/10/1955 17:00	<NA>	circle	53.2	-2.916667
4	10/10/1956 21:00	tx	circle	28.9783333	-96.6458333
5	10/10/1960 20:00	hi	light	21.4180556	-157.8036111
6	10/10/1961 19:00	tn	sphere	36.5950000	-82.1888889
7	10/10/1965 21:00	<NA>	circle	51.434722	-3.18
8	10/10/1965 23:45	ct	disk	41.1175000	-73.4083333
9	10/10/1966 20:00	al	disk	33.5861111	-86.2861111
10	10/10/1966 21:00	fl	disk	30.2947222	-82.9841667

```
# i 88,865 more rows
```

Tidymodel helpers

We are going to cover “fancier” ways of matching column names (and strings more generally) in the data cleaning lecture.

Subset rows of a `data.frame`:

The command in `dplyr` for subsetting rows is `filter`. Try `?filter`.

The easiest way to filter is by testing whether numeric observations are greater than or less than some cutoff:

```
filter(ufo, `duration (seconds)` > 6000)
```

```
# A tibble: 2,839 × 11
  datetime      city      state country shape `duration (seconds)`
  <chr>         <chr>    <chr> <chr>   <chr>         <dbl>
1 10/10/1949 21:00 lackland afb tx    <NA>   light          7200
2 10/10/1997 16:00 connersville in     us     delta         14400
3 10/10/1998 22:30 st. johns (can... nf     ca     egg           7200
4 10/10/1999 21:00 rachel      nv     us     light         10800
5 10/10/2003 20:25 temperance  mi     us     oval          18000
6 10/10/2003 21:10 crescent beach sc     us     form...       37800
7 10/10/2004 21:00 mansfield  oh     us     light          7200
8 10/10/2007 01:00 lebanon     or     us     light         14400
9 10/10/2007 13:00 owego       ny     us     tria...       10800
10 10/10/2010 01:00 orchard park ny     us     light          7200
# i 2,829 more rows
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
# `date posted` <chr>, latitude <chr>, longitude <chr>
```

Subset using pipes:

You can also using piping here:

```
ufo %>% filter(`duration (seconds)` > 6000)
```

```
# A tibble: 2,839 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1949 21:00	lackland afb	tx	<NA>	light	7200
2	10/10/1997 16:00	connersville	in	us	delta	14400
3	10/10/1998 22:30	st. johns (can...	nf	ca	egg	7200
4	10/10/1999 21:00	rachel	nv	us	light	10800
5	10/10/2003 20:25	temperance	mi	us	oval	18000
6	10/10/2003 21:10	crescent beach	sc	us	form...	37800
7	10/10/2004 21:00	mansfield	oh	us	light	7200
8	10/10/2007 01:00	lebanon	or	us	light	14400
9	10/10/2007 13:00	owego	ny	us	tria...	10800
10	10/10/2010 01:00	orchard park	ny	us	light	7200

```
# i 2,829 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date posted` <chr>, latitude <chr>, longitude <chr>
```


Subset rows for a character condition:

You can also filter character strings by a single value or category. Here we need quotes around character strings.

```
ufo %>% filter(country == "us")
```

```
# A tibble: 70,293 × 11
```

	datetime <chr>	city <chr>	state <chr>	country <chr>	shape <chr>	`duration (seconds)` <dbl>
1	10/10/1949 20:30	san marcos	tx	us	cylinder	2700
2	10/10/1956 21:00	edna	tx	us	circle	20
3	10/10/1960 20:00	kaneohe	hi	us	light	900
4	10/10/1961 19:00	bristol	tn	us	sphere	300
5	10/10/1965 23:45	norwalk	ct	us	disk	1200
6	10/10/1966 20:00	pell city	al	us	disk	180
7	10/10/1966 21:00	live oak	fl	us	disk	120
8	10/10/1968 13:00	hawthorne	ca	us	circle	300
9	10/10/1968 19:00	brevard	nc	us	fireball	180
10	10/10/1970 16:00	bellmore	ny	us	disk	1800

```
# i 70,283 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date posted` <chr>, latitude <chr>, longitude <chr>
```

Subset rows with multiple required conditions:

You can combine filtering on multiple columns by separating the filter arguments with & (or commas - but & is more clear) - must match both conditions!

```
ufo %>% filter(`duration (seconds)` > 6000 & country == "us")
```

```
# A tibble: 2,164 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1997 16:00	connersville	in	us	delta	14400
2	10/10/1999 21:00	rachel	nv	us	light	10800
3	10/10/2003 20:25	temperance	mi	us	oval	18000
4	10/10/2003 21:10	crescent beach	sc	us	formation	37800
5	10/10/2004 21:00	mansfield	oh	us	light	7200
6	10/10/2007 01:00	lebanon	or	us	light	14400
7	10/10/2007 13:00	owego	ny	us	triangle	10800
8	10/10/2010 01:00	orchard park	ny	us	light	7200
9	10/10/2010 21:00	kykotsmovi	az	us	disk	7200
10	10/10/2010 22:30	garden grove	ca	us	egg	7200

```
# i 2,154 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date posted` <chr>, latitude <chr>, longitude <chr>
```

Subset to match a vector of possibilities:

Sometimes you want to be able to filter on matching several values or categories. The `%in%` operator is useful here:

```
ufo %>% filter(country %in% c("us", "gb"))
```

```
# A tibble: 72,343 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1949 20:30	san marcos	tx	us	cyli...	2700
2	10/10/1955 17:00	chester (uk/englan...	<NA>	gb	circ...	20
3	10/10/1956 21:00	edna	tx	us	circ...	20
4	10/10/1960 20:00	kaneohe	hi	us	light	900
5	10/10/1961 19:00	bristol	tn	us	sphe...	300
6	10/10/1965 21:00	penarth (uk/wales)	<NA>	gb	circ...	180
7	10/10/1965 23:45	norwalk	ct	us	disk	1200
8	10/10/1966 20:00	pell city	al	us	disk	180
9	10/10/1966 21:00	live oak	fl	us	disk	120
10	10/10/1968 13:00	hawthorne	ca	us	circ...	300

```
# i 72,333 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date posted` <chr>, latitude <chr>, longitude <chr>
```

Subset rows using conditions for numeric and character variables:

You can mix and match filtering on numeric and categorical/character columns in the same `filter()` command:

```
ufo %>% filter(country %in% c("us", "gb") &
               `duration (seconds)` > 6000)
```

```
# A tibble: 2,208 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1997 16:00	connersville	in	us	delta	14400
2	10/10/1999 21:00	rachel	nv	us	light	10800
3	10/10/2003 20:25	temperance	mi	us	oval	18000
4	10/10/2003 21:10	crescent beach	sc	us	formation	37800
5	10/10/2004 21:00	mansfield	oh	us	light	7200
6	10/10/2007 01:00	lebanon	or	us	light	14400
7	10/10/2007 13:00	owego	ny	us	triangle	10800
8	10/10/2010 01:00	orchard park	ny	us	light	7200
9	10/10/2010 21:00	kykotsmovi	az	us	disk	7200
10	10/10/2010 22:30	garden grove	ca	us	egg	7200

```
# i 2,198 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date posted` <chr>, latitude <chr>, longitude <chr>
```

Note about quotes and numbers

R will interpret quotes around numbers as the characters themselves and not their numeric meaning. Avoid quotes unless it is not being treated as a numeric value - for example grades.

```
ufo %>% filter(`duration (seconds)` > 60000) %>% head(2) #This works
```

```
# A tibble: 2 × 11
  datetime city state country shape `duration (seconds)` `duration (hours/min)`
  <chr>    <chr> <chr> <chr>   <chr>                <dbl> <chr>
1 10/11/2... san ... ca    us    <NA>                172800 2 days?
2 10/1/19... birm... <NA> gb    sphe...            97836000 31 years
# i 4 more variables: comments <chr>, `date posted` <chr>, latitude <chr>,
# longitude <chr>
```

```
ufo %>% filter(`duration (seconds)` > "60000") %>% head(2) # not right!
```

```
# A tibble: 2 × 11
  datetime city state country shape `duration (seconds)` `duration (hours/min)`
  <chr>    <chr> <chr> <chr>   <chr>                <dbl> <chr>
1 10/10/1... lack... tx    <NA>    light                7200 1-2 hrs
2 10/10/1... kane... hi    us    light                900 15 minutes
# i 4 more variables: comments <chr>, `date posted` <chr>, latitude <chr>,
# longitude <chr>
```

Watch out for quotes and column names

```
ufo %>% filter("country" == "gb") # didn't work!
```

```
# A tibble: 0 × 11
#   i 11 variables: datetime <chr>, city <chr>, state <chr>, country <chr>,
#   shape <chr>, duration (seconds) <dbl>, duration (hours/min) <chr>,
#   comments <chr>, date posted <chr>, latitude <chr>, longitude <chr>
```

```
ufo %>% filter(country == "gb")
```

```
# A tibble: 2,050 × 11
  datetime          city          state country shape `duration (seconds)`
  <chr>            <chr>          <chr> <chr>   <chr>          <dbl>
1 10/10/1955 17:00 chester (uk/englan... <NA>  gb      circ...          20
2 10/10/1965 21:00 penarth (uk/wales) <NA>  gb      circ...         180
3 10/10/1974 21:30 cardiff (uk/wales) <NA>  gb      disk          1200
4 10/10/1976 22:00 stoke mandeville (... <NA>  gb      cigar           3
5 10/10/1985 20:25 leeds (uk/england) <NA>  gb      tria...         600
6 10/10/2005 20:00 newtown (uk/wales) <NA>  gb      form...         360
7 10/10/2006 21:47 plymouth (devonshi... <NA>  gb      form...         120
8 10/10/2011 14:00 epsom (surrey) (uk... <NA>  gb      oval           300
9 10/11/2000 19:30 london (uk/england) <NA>  gb      diam...           3
10 10/11/2000 22:09 dolgellau (uk/wale... <NA>  gb      chan...        240
# i 2,040 more rows
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
#   `date posted` <chr>, latitude <chr>, longitude <chr>
```

Special column names need back ticks

Quotes work too but this can cause issues. The backtick key is usually near the top left for standard US keyboards, under the escape key. May need to press shift or Fn key if you have a smaller keyboard.



```
ufo %>% filter(duration (seconds) > 6000) # didn't work!
```

```
Error in `filter()`:
```

```
! In argument: `duration(seconds) > 6000`.
```

```
Caused by error in `duration_from_num()`:
```

```
! First argument to `duration()` constructor must be character or numeric. Supplied object of class 'function'
```

```
ufo %>% filter(`duration (seconds)` > 6000) # worked!
```

```
# A tibble: 2,839 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1949 21:00	lackland afb	tx	<NA>	light	7200
2	10/10/1997 16:00	connersville	in	us	delta	14400
3	10/10/1998 22:30	st. johns (can...	nf	ca	egg	7200
4	10/10/1999 21:00	rachel	nv	us	light	10800
5	10/10/2003 20:25	temperance	mi	us	oval	18000
6	10/10/2003 21:10	crescent beach	sc	us	form...	37800
7	10/10/2004 21:00	mansfield	oh	us	light	7200

Logic for subsetting rows of a **data.frame**:

Other useful logical tests:

& : AND

| : OR

<= : less than or equals

>= : greater than or equals

!= : not equals

Subset rows with or logic:

The OR operator (|) is more permissive than the AND operator:

```
ufo %>% filter(country == "gb") %>% dim()
```

```
[1] 2050  11
```

```
ufo %>% filter(country == "gb" | `duration (seconds)` > 6000) %>% dim()
```

```
[1] 4845  11
```

```
ufo %>% filter(country == "gb" | `duration (seconds)` > 6000)
```

```
# A tibble: 4,845 × 11
```

	datetime	city	state	country	shape	`duration (seconds)`
	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>
1	10/10/1949 21:00	lackland afb	tx	<NA>	light	7200
2	10/10/1955 17:00	chester (uk/englan...	<NA>	gb	circ...	20
3	10/10/1965 21:00	penarth (uk/wales)	<NA>	gb	circ...	180
4	10/10/1974 21:30	cardiff (uk/wales)	<NA>	gb	disk	1200
5	10/10/1976 22:00	stoke mandeville (...)	<NA>	gb	cigar	3
6	10/10/1985 20:25	leeds (uk/england)	<NA>	gb	tria...	600
7	10/10/1997 16:00	connersville	in	us	delta	14400
8	10/10/1998 22:30	st. johns (can...	nf	ca	egg	7200
9	10/10/1999 21:00	rachel	nv	us	light	10800
10	10/10/2003 20:25	temperance	mi	us	oval	18000

```
# i 4,835 more rows
```

```
# i 5 more variables: `duration (hours/min)` <chr>, comments <chr>,
```

```
# `date_posted` <chr>, latitude <chr>, longitude <chr>
```

OR is like `%in%`:

The OR operator (`|`) can be a substitute for `%in%` (although it might take more typing):

```
ufo %>% filter(country == "us" | country == "gb") %>% dim()
```

```
[1] 72343    11
```

```
ufo %>% filter(country %in% c("us", "gb")) %>% dim()
```

```
[1] 72343    11
```

Combining `filter` and `select`:

You can combine `filter` and `select` to subset the rows and columns, respectively, of a `data.frame`:

```
ufo%>%  
  filter(country == "gb") %>%  
  select(starts_with("c"))
```

```
# A tibble: 2,050 × 3
```

city	country	comments
<chr>	<chr>	<chr>
1 chester (uk/england)	gb	Green/Orange circular disc over C...
2 penarth (uk/wales)	gb	penarth uk circle 3mins stayed...
3 cardiff (uk/wales)	gb	back in 1974 I was 19 at the time...
4 stoke mandeville (uk/england)	gb	White object over Buckinghamshire...
5 leeds (uk/england)	gb	three light in the sky that led t...
6 newtown (uk/wales)	gb	red/orang lights dancing in the s...
7 plymouth (devonshire) (uk/england)	gb	they were right above me very sma...
8 epsom (surrey) (uk/england)	gb	Flying beer barrel shaped metalli...
9 london (uk/england)	gb	Diamond shaped ufo seen pulsing f...
10 dolgellau (uk/wales)	gb	How two became one.

```
# i 2,040 more rows
```

Combining **filter** and **select**- Order matters!:

The order of these functions matters though, since you can remove columns that you might want to filter on.

```
ufo%>%  
  select(starts_with("c")) %>%  
  filter(shape == "light")
```

```
Error in `filter()`:  
i In argument: `shape == "light"`.  
Caused by error:  
! object 'shape' not found
```

This will result in an error because the table column is now gone after the `select()` function!

Fancier filtering

Combining tidysselect helpers with regular selection

```
head(ufo, 2)
```

```
# A tibble: 2 × 11
  datetime city state country shape `duration (seconds)` `duration (hours/min)`
  <chr>    <chr> <chr> <chr>   <chr>          <dbl> <chr>
1 10/10/1... san ... tx    us     cyli...      2700 45 minutes
2 10/10/1... lack... tx    <NA>    light      7200 1-2 hrs
# i 4 more variables: comments <chr>, `date posted` <chr>, latitude <chr>,
# longitude <chr>
```

```
ufo %>% select(`duration (seconds)`, starts_with("c"))
```

```
# A tibble: 88,875 × 4
  `duration (seconds)` city country comments
  <dbl> <chr>          <chr> <chr>
1      2700 san marcos    us    This event took place in e...
2      7200 lackland afb <NA>    1949 Lackland AFB #44 TX. ...
3         20 chester (uk/england) gb    Green/Orange circular disc...
4         20 edna        us    My older brother and twin ...
5        900 kaneohe     us    AS a Marine 1st Lt. flying...
6        300 bristol     us    My father is now 89 my bro...
7        180 penarth (uk/wales) gb    penarth uk circle 3mins ...
8       1200 norwalk     us    A bright orange color chan...
9        180 pell city   us    Strobe Lighted disk shape ...
10       120 live oak    us    Saucer zaps energy from po...
# i 88,865 more rows
```

Multiple tidyselect functions

Follows OR logic.

```
ufo %>% select(starts_with("c"), ends_with("e"))
```

```
# A tibble: 88,875 × 8
```

	city	country	comments	datetime	state	shape	latitude	longitude
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	san marcos	us	This ev...	10/10/1...	tx	cyli...	29.8830...	-97.9411...
2	lackland afb	<NA>	1949 La...	10/10/1...	tx	light	29.38421	-98.5810...
3	chester (uk/england)	gb	Green/0...	10/10/1...	<NA>	circ...	53.2	-2.916667
4	edna	us	My olde...	10/10/1...	tx	circ...	28.9783...	-96.6458...
5	kaneohe	us	AS a Ma...	10/10/1...	hi	light	21.4180...	-157.803...
6	bristol	us	My fath...	10/10/1...	tn	sphe...	36.5950...	-82.1888...
7	penarth (uk/wales)	gb	penarth...	10/10/1...	<NA>	circ...	51.4347...	-3.18
8	norwalk	us	A brigh...	10/10/1...	ct	disk	41.1175...	-73.4083...
9	pell city	us	Strobe ...	10/10/1...	al	disk	33.5861...	-86.2861...
10	live oak	us	Saucer ...	10/10/1...	fl	disk	30.2947...	-82.9841...

```
# i 88,865 more rows
```

Multiple patterns with tidyselect

Need to combine the patterns with the `c()` function.

```
ufo %>% select(starts_with(c("c", "s")))
```

```
# A tibble: 88,875 × 5
```

	city	country	comments	state	shape
	<chr>	<chr>	<chr>	<chr>	<chr>
1	san marcos	us	This event took place in early fall...	tx	cyli...
2	lackland afb	<NA>	1949 Lackland AFB#44 TX. Lights r...	tx	light
3	chester (uk/england)	gb	Green/Orange circular disc over Che...	<NA>	circ...
4	edna	us	My older brother and twin sister we...	tx	circ...
5	kaneohe	us	AS a Marine 1st Lt. flying an FJ4B ...	hi	light
6	bristol	us	My father is now 89 my brother 52 t...	tn	sphe...
7	penarth (uk/wales)	gb	penarth uk circle 3mins stayed 3...	<NA>	circ...
8	norwalk	us	A bright orange color changing to r...	ct	disk
9	pell city	us	Strobe Lighted disk shape object ob...	al	disk
10	live oak	us	Saucer zaps energy from powerline a...	fl	disk

```
# i 88,865 more rows
```


Common error for filter or select

If you try to filter or select for a column that does not exist it will not work:

- misspelled column name
- column that was already removed
- using quotes when you shouldn't for numeric values

Always good to check each step!

Did the filter work the way you expected? Did the dimensions change?

Don't use quotes on column names - nothing will get filtered!



Source: <https://media.giphy.com/media/5b5OU7aUekfdSAER5I/giphy.gif>

Summary

- `pull()` can help us see a vector version of our variables - we can “pull” out the data from a dataframe
- The pipe (`%>%` or `%>%`) can help us to do sequential steps
- `select()` makes a smaller table of just selected variables
- tidyselect functions can help us select specific columns: `contains()`, `ends_with()`, `starts_with()`
- `filter` can remove rows based on conditions
- `==` is needed to filter for rows that are “exactly equal” to a value
- `!=` does the opposite

Summary Continued

- `%in%` enables us to do multiple `==` conditions such as `%in% c(1,2,3)`
- `|` is for or logic and `&` is for and logic when combining filter conditions together
- Always check that you filtered for what you think you did!

Lab

[Link to Lab](#)